

Lab Assignment 03 — Cats vs Dogs: Transfer Learning CNN Comparison

End-to-end pipeline: EDA, preprocessing, augmentation, 5 pretrained backbones, feature extraction, fine-tuning, hyperparameter tuning, evaluation, error analysis, and export for deployment.

All figures and tables produced by this notebook are automatically saved as PDF files in an `outputs/` folder, and the folder is zipped at the end for one-click download.

1. Setup & Dataset Download

```
In [1]: !pip install -q kagglehub

import kagglehub
import os, glob, time, json, random, shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers, callbacks
from PIL import Image
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_curve, auc,
                             precision_score, recall_score, f1_score,
                             accuracy_score)

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)

# Download latest version
path = kagglehub.dataset_download("samuelcortinhas/cats-and-dogs-image-
classification")
print("Path to dataset files:", path)

# The dataset root usually contains train/ and test/ each with cats/ and
dogs/
train_dir = os.path.join(path, "train")
test_dir = os.path.join(path, "test")
print(os.listdir(path))
```

Using Colab cache for faster access to the 'cats-and-dogs-image-classification' dataset.
Path to dataset files: /kaggle/input/cats-and-dogs-image-classification
['test', 'train']

1b. Output Folder & PDF-Saving Helpers

Every figure and table below is written as a `.pdf` into `OUTPUT_DIR` so the whole run leaves a self-contained set of report-ready PDFs.

```

In [2]: OUTPUT_DIR = "/content/outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

def save_fig(fig, filename):
    """Save a matplotlib figure as a PDF inside OUTPUT_DIR."""
    if not filename.endswith(".pdf"):
        filename += ".pdf"
    out_path = os.path.join(OUTPUT_DIR, filename)
    fig.savefig(out_path, format="pdf", bbox_inches="tight")
    print("Saved:", out_path)
    return out_path

def save_df_as_pdf(df, filename, title=None):
    """Render a DataFrame as a table and save it as a PDF inside
    OUTPUT_DIR."""
    n_rows, n_cols = len(df), len(df.columns)
    fig, ax = plt.subplots(figsize=(max(6, 1.6 * n_cols), 0.9 + 0.45 *
(n_rows + 1)))
    ax.axis("off")
    if title:
        ax.set_title(title, fontsize=13, weight="bold", pad=14)
    tbl = ax.table(cellText=df.values, colLabels=df.columns, loc="center",
cellLoc="center")
    tbl.auto_set_font_size(False)
    tbl.set_fontsize(9)
    tbl.scale(1, 1.5)
    tbl.auto_set_column_width(col=list(range(n_cols)))
    path = save_fig(fig, filename)
    plt.close(fig)
    return path

def save_text_as_pdf(text, filename, title=None):
    """Render a block of text (e.g. a classification report) as a PDF
    page."""
    fig = plt.figure(figsize=(8.5, 11))
    y = 0.95
    if title:
        fig.text(0.06, y, title, fontsize=14, weight="bold", va="top")
        y -= 0.05
    fig.text(0.06, y, text, fontsize=10, va="top", family="monospace")
    plt.axis("off")
    path = save_fig(fig, filename)
    plt.close(fig)
    return path

```

2. Task 1 — Dataset Exploration

```

In [3]: def scan_dir(root):
        rows = []
        for cls in ["cats", "dogs"]:
            cls_dir = os.path.join(root, cls)

```

```

        if not os.path.isdir(cls_dir):
            continue
        for fp in glob.glob(os.path.join(cls_dir, "*")):
            rows.append({"path": fp, "class": cls})
    return pd.DataFrame(rows)

df_train = scan_dir(train_dir)
df_test = scan_dir(test_dir)
print(f"Train images: {len(df_train)}    Test images: {len(df_test)}")
print("\nClass balance (train):\n", df_train["class"].value_counts())
print("\nClass balance (test):\n", df_test["class"].value_counts())

def image_stats(df, sample=None):
    widths, heights, modes, corrupted = [], [], [], []
    rows = df.sample(sample, random_state=SEED) if sample else df
    for fp in rows["path"]:
        try:
            with Image.open(fp) as im:
                w, h = im.size
                widths.append(w); heights.append(h); modes.append(im.mode)
        except Exception as e:
            corrupted.append(fp)
    return widths, heights, modes, corrupted

widths, heights, modes, corrupted = image_stats(df_train)
print(f"\nMin dims: {min(widths)}x{min(heights)}    Max dims: {max(widths)}x{max(heights)}")
print(f"Avg dims: {np.mean(widths):.0f}x{np.mean(heights):.0f}")
print(f"RGB fraction: {sum(m == 'RGB' for m in modes) / len(modes):.2%}")
print(f"Corrupted/unreadable images found: {len(corrupted)}")
if corrupted:
    print(corrupted[:10])

eda_summary = pd.DataFrame([
    "Train images": len(df_train), "Test images": len(df_test),
    "Train cats": int((df_train["class"] == "cats").sum()), "Train dogs":
int((df_train["class"] == "dogs").sum()),
    "Min W x H": f"{min(widths)}x{min(heights)}", "Max W x H": f"
{max(widths)}x{max(heights)}",
    "Avg W x H": f"{np.mean(widths):.0f}x{np.mean(heights):.0f}",
    "RGB fraction": f"{sum(m=='RGB' for m in modes)/len(modes):.2%}",
    "Corrupted images": len(corrupted),
])
save_df_as_pdf(eda_summary, "01_dataset_summary", title="Dataset Exploration
Summary")

```

Train images: 557 Test images: 140

Class balance (train):

class	
cats	279
dogs	278

```
Name: count, dtype: int64

Class balance (test):
  class
cats    70
dogs    70
Name: count, dtype: int64

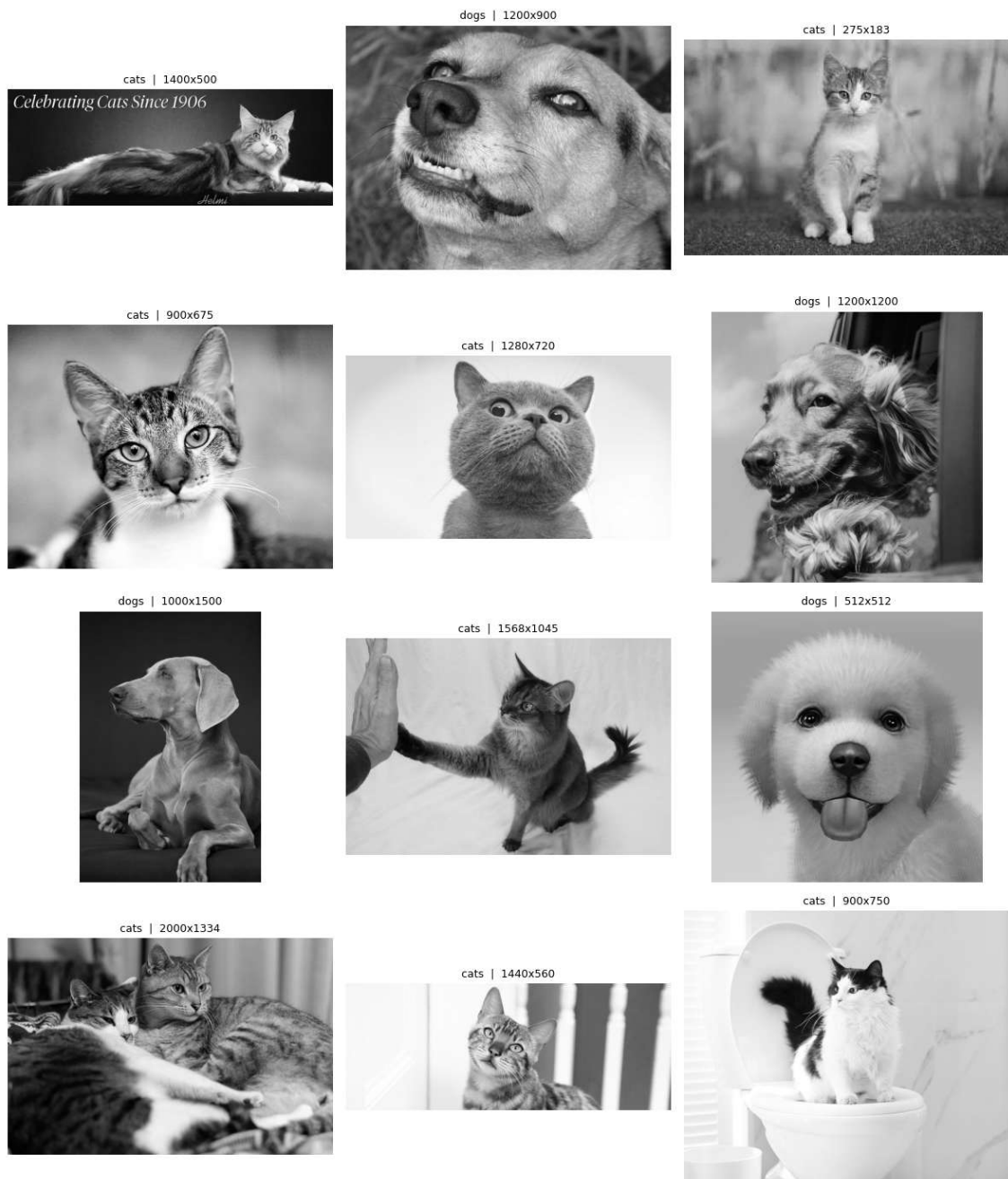
Min dims: 133x133  Max dims: 4272x4272  Avg dims: 952x657
RGB fraction: 99.82%
Corrupted/unreadable images found: 0
Saved: /content/outputs/01_dataset_summary.pdf

out[3]:  '/content/outputs/01_dataset_summary.pdf'
```

3. Task 2 — Visual Exploration (12-image grid)

```
In [4]: sample_df = df_train.sample(12, random_state=SEED).reset_index(drop=True)
fig, axes = plt.subplots(4, 3, figsize=(12, 14))
for ax, (_, row) in zip(axes.flatten(), sample_df.iterrows()):
    img = Image.open(row["path"])
    ax.imshow(img)
    ax.set_title(f'{row["class"]} | {img.size[0]}x{img.size[1]}',
    fontsize=9)
    ax.axis("off")
plt.tight_layout()
save_fig(fig, "02_sample_image_grid")
plt.show()
```

Saved: /content/outputs/02_sample_image_grid.pdf



4. Image Preprocessing (resize 128×128×3, normalize)

Using `image_dataset_from_directory` which handles batched loading, resizing and labels. Normalization is applied as a `Rescaling` layer inside the model pipeline so it is applied consistently at train **and** inference time.

```
In [5]: IMG_SIZE = (128, 128)
        BATCH_SIZE = 32

        raw_train_ds = tf.keras.utils.image_dataset_from_directory(
            train_dir, labels="inferred", label_mode="binary", class_names=["cats",
            "dogs"],
            image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True, seed=SEED,
            validation_split=0.15, subset="training")

        val_ds = tf.keras.utils.image_dataset_from_directory(
            train_dir, labels="inferred", label_mode="binary", class_names=["cats",
            "dogs"],
            image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True, seed=SEED,
            validation_split=0.15, subset="validation")

        test_ds = tf.keras.utils.image_dataset_from_directory(
            test_dir, labels="inferred", label_mode="binary", class_names=["cats",
            "dogs"],
            image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False)

        AUTOTUNE = tf.data.AUTOTUNE
        raw_train_ds = raw_train_ds.prefetch(AUTOTUNE)
        val_ds = val_ds.prefetch(AUTOTUNE)
        test_ds = test_ds.prefetch(AUTOTUNE)

        rescale = layers.Rescaling(1.0 / 255) # [0,255] → [0,1], applied inside
        each model's forward pass
```

```
Found 557 files belonging to 2 classes.
Using 474 files for training.
Found 557 files belonging to 2 classes.
Using 83 files for validation.
Found 140 files belonging to 2 classes.
```

5. Data Augmentation (train-only)

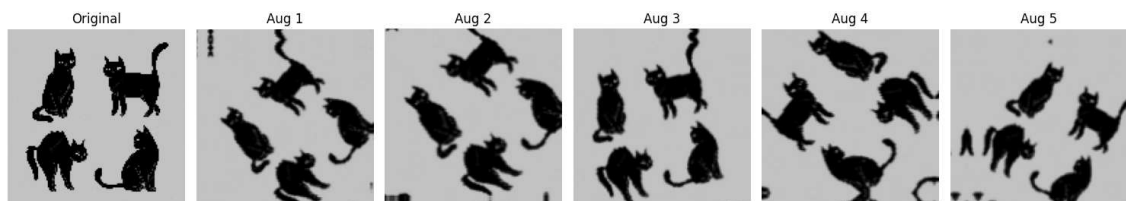
```
In [6]: data_augmentation = tf.keras.Sequential([
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.15),
        layers.RandomZoom(0.15),
        layers.RandomTranslation(0.1, 0.1),
    ], name="augmentation")

train_ds = raw_train_ds.map(lambda x, y: (data_augmentation(x,
                                                                    training=True), y),
                           num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)

# Visualize: one image, 5 augmented versions
sample_batch = next(iter(raw_train_ds.take(1)))
sample_img = sample_batch[0][0]

fig = plt.figure(figsize=(15, 3))
plt.subplot(1, 6, 1); plt.imshow(sample_img.numpy().astype("uint8"));
plt.title("Original"); plt.axis("off")
for i in range(5):
    aug = data_augmentation(tf.expand_dims(sample_img, 0), training=True)[0]
    plt.subplot(1, 6, i + 2); plt.imshow(aug.numpy().astype("uint8"));
    plt.title(f"Aug {i+1}"); plt.axis("off")
plt.tight_layout()
save_fig(fig, "03_augmentation_examples")
plt.show()
```

Saved: /content/outputs/03_augmentation_examples.pdf



Discussion (fill in with your own words for the report):

1. Augmentation increases diversity by synthesizing new viewpoints/lighting/positions from existing images, so the model sees more variation without collecting new data.
2. It reduces overfitting by preventing the model from memorizing exact pixel arrangements of the training set.
3. It's applied only to training data because validation/test sets must reflect real, unaltered inputs to give an unbiased estimate of generalization.
4. Overly aggressive augmentation can distort images beyond realistic variation (e.g. unrecognizable shapes), which can hurt rather than help learning.

6. Transfer Learning Model Builder

```
In [7]: from tensorflow.keras.applications import vgg16, resnet50, mobilenet_v2,
        efficientnet, xception

MODEL_REGISTRY = {
    "VGG16": dict(app=tf.keras.applications.VGG16,
preprocess=vgg16.preprocess_input),
    "ResNet50": dict(app=tf.keras.applications.ResNet50,
preprocess=resnet50.preprocess_input),
    "MobileNetV2": dict(app=tf.keras.applications.MobileNetV2,
preprocess=mobilenet_v2.preprocess_input),
    "EfficientNetB0": dict(app=tf.keras.applications.EfficientNetB0,
preprocess=efficientnet.preprocess_input),
    "Xception": dict(app=tf.keras.applications.Xception,
preprocess=xception.preprocess_input),
}

def build_model(name, input_shape=(128, 128, 3), dropout=0.3,
dense_units=128, trainable_base=False):
    cfg = MODEL_REGISTRY[name]
    base = cfg["app"](include_top=False, weights="imagenet",
input_shape=input_shape)
    base.trainable = trainable_base

    inputs = layers.Input(shape=input_shape)
    x = layers.Lambda(cfg["preprocess"], name="preprocess")(inputs)    # each
model's own preprocessing
    x = base(x, training=trainable_base)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(dense_units, activation="relu")(x)
    x = layers.Dropout(dropout)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = models.Model(inputs, outputs, name=name)
    return model, base
```

7. Feature-Extraction Training (base frozen) — all 5 models

```
In [8]: EPOCHS_FE = 12

def compile_model(model, lr=1e-3):
    model.compile(optimizer=optimizers.Adam(learning_rate=lr),
                  loss="binary_crossentropy",
                  metrics=["accuracy", tf.keras.metrics.AUC(name="auc")])
    return model

results = {}
histories = {}
trained_models = {}

for name in MODEL_REGISTRY:
    print(f"\n==== Training {name} (feature extraction) =====")
    model, base = build_model(name, trainable_base=False)
    compile_model(model)

    es = callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                restore_best_weights=True)
    start = time.time()
    hist = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS_FE,
                    callbacks=[es], verbose=1)
    train_time = time.time() - start

    trained_models[name] = model
    histories[name] = hist.history
    results[name] = {
        "params": model.count_params(),
        "trainable_params": sum(tf.size(w).numpy() for w in
                                model.trainable_weights),
        "train_time_sec": round(train_time, 1),
    }
    print(f"{name}: {results[name]}")
```

```
==== Training VGG16 (feature extraction) =====
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ————— 1s 0us/step
Epoch 1/12
15/15 ————— 27s 995ms/step - accuracy: 0.6793 - auc: 0.7487 -
loss: 1.9305 - val_accuracy: 0.8434 - val_auc: 0.9269 - val_loss: 0.5724
Epoch 2/12
15/15 ————— 4s 232ms/step - accuracy: 0.8080 - auc: 0.8787 -
loss: 0.9053 - val_accuracy: 0.8916 - val_auc: 0.9513 - val_loss: 0.4687
Epoch 3/12
15/15 ————— 4s 233ms/step - accuracy: 0.8586 - auc: 0.9242 -
loss: 0.5973 - val_accuracy: 0.9036 - val_auc: 0.9656 - val_loss: 0.3184
Epoch 4/12
15/15 ————— 6s 326ms/step - accuracy: 0.8797 - auc: 0.9396 -
loss: 0.4546 - val_accuracy: 0.8795 - val_auc: 0.9749 - val_loss: 0.2743
```

```

Epoch 5/12
15/15 ————— 4s 222ms/step - accuracy: 0.8713 - auc: 0.9474 -
loss: 0.3925 - val_accuracy: 0.8675 - val_auc: 0.9645 - val_loss: 0.2985
Epoch 6/12
15/15 ————— 5s 231ms/step - accuracy: 0.8861 - auc: 0.9585 -
loss: 0.3163 - val_accuracy: 0.9157 - val_auc: 0.9679 - val_loss: 0.2561
Epoch 7/12
15/15 ————— 5s 226ms/step - accuracy: 0.9198 - auc: 0.9748 -
loss: 0.2360 - val_accuracy: 0.8795 - val_auc: 0.9793 - val_loss: 0.3528
Epoch 8/12
15/15 ————— 4s 225ms/step - accuracy: 0.8924 - auc: 0.9684 -
loss: 0.2776 - val_accuracy: 0.8795 - val_auc: 0.9595 - val_loss: 0.3148
Epoch 9/12
15/15 ————— 4s 257ms/step - accuracy: 0.9262 - auc: 0.9762 -
loss: 0.2103 - val_accuracy: 0.8675 - val_auc: 0.9610 - val_loss: 0.3688
VGG16: {'params': 14780481, 'trainable_params': np.int32(65793),
'train_time_sec': 63.3}

```

==== Training ResNet50 (feature extraction) =====

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ————— 1s 0us/step

```

Epoch 1/12
15/15 ————— 30s 1s/step - accuracy: 0.7468 - auc: 0.8523 -
loss: 0.5856 - val_accuracy: 0.9157 - val_auc: 0.9575 - val_loss: 0.2765
Epoch 2/12
15/15 ————— 4s 268ms/step - accuracy: 0.8861 - auc: 0.9518 -
loss: 0.2976 - val_accuracy: 0.9277 - val_auc: 0.9627 - val_loss: 0.2680
Epoch 3/12
15/15 ————— 5s 236ms/step - accuracy: 0.8903 - auc: 0.9698 -
loss: 0.2235 - val_accuracy: 0.8795 - val_auc: 0.9717 - val_loss: 0.2556
Epoch 4/12
15/15 ————— 4s 238ms/step - accuracy: 0.9283 - auc: 0.9829 -
loss: 0.1631 - val_accuracy: 0.9518 - val_auc: 0.9621 - val_loss: 0.2155
Epoch 5/12
15/15 ————— 6s 307ms/step - accuracy: 0.9283 - auc: 0.9774 -
loss: 0.1917 - val_accuracy: 0.9518 - val_auc: 0.9682 - val_loss: 0.2238
Epoch 6/12
15/15 ————— 4s 236ms/step - accuracy: 0.9283 - auc: 0.9782 -
loss: 0.1850 - val_accuracy: 0.9157 - val_auc: 0.9674 - val_loss: 0.2112
Epoch 7/12
15/15 ————— 4s 223ms/step - accuracy: 0.9578 - auc: 0.9863 -
loss: 0.1447 - val_accuracy: 0.8795 - val_auc: 0.9662 - val_loss: 0.2225
Epoch 8/12
15/15 ————— 5s 301ms/step - accuracy: 0.9494 - auc: 0.9948 -
loss: 0.1015 - val_accuracy: 0.9036 - val_auc: 0.9674 - val_loss: 0.2161
Epoch 9/12
15/15 ————— 4s 234ms/step - accuracy: 0.9388 - auc: 0.9911 -
loss: 0.1236 - val_accuracy: 0.8916 - val_auc: 0.9776 - val_loss: 0.2082
Epoch 10/12
15/15 ————— 4s 225ms/step - accuracy: 0.9599 - auc: 0.9929 -
loss: 0.1065 - val_accuracy: 0.8916 - val_auc: 0.9706 - val_loss: 0.2377
Epoch 11/12
15/15 ————— 4s 264ms/step - accuracy: 0.9684 - auc: 0.9945 -

```

```

loss: 0.0903 - val_accuracy: 0.9277 - val_auc: 0.9729 - val_loss: 0.2222
Epoch 12/12
15/15 ————— 4s 247ms/step - accuracy: 0.9684 - auc: 0.9945 -
loss: 0.0932 - val_accuracy: 0.8916 - val_auc: 0.9712 - val_loss: 0.2310
ResNet50: {'params': 23850113, 'trainable_params': np.int32(262401),
'train_time_sec': 76.5}

```

==== Training MobileNetV2 (feature extraction) =====

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_128_no_top.h5

```

9406464/9406464 ————— 0s 0us/step

```

Epoch 1/12

```

15/15 ————— 55s 3s/step - accuracy: 0.7595 - auc: 0.8652 -
loss: 0.4561 - val_accuracy: 0.9277 - val_auc: 0.9764 - val_loss: 0.1921

```

Epoch 2/12

```

15/15 ————— 5s 313ms/step - accuracy: 0.8945 - auc: 0.9601 -
loss: 0.2480 - val_accuracy: 0.9398 - val_auc: 0.9767 - val_loss: 0.1734

```

Epoch 3/12

```

15/15 ————— 4s 229ms/step - accuracy: 0.9325 - auc: 0.9798 -
loss: 0.1826 - val_accuracy: 0.9518 - val_auc: 0.9814 - val_loss: 0.1547

```

Epoch 4/12

```

15/15 ————— 4s 240ms/step - accuracy: 0.9262 - auc: 0.9812 -
loss: 0.1771 - val_accuracy: 0.9518 - val_auc: 0.9799 - val_loss: 0.1633

```

Epoch 5/12

```

15/15 ————— 5s 332ms/step - accuracy: 0.9557 - auc: 0.9901 -
loss: 0.1326 - val_accuracy: 0.9398 - val_auc: 0.9851 - val_loss: 0.1496

```

Epoch 6/12

```

15/15 ————— 4s 221ms/step - accuracy: 0.9473 - auc: 0.9905 -
loss: 0.1281 - val_accuracy: 0.9398 - val_auc: 0.9811 - val_loss: 0.1515

```

Epoch 7/12

```

15/15 ————— 5s 217ms/step - accuracy: 0.9536 - auc: 0.9882 -
loss: 0.1400 - val_accuracy: 0.9277 - val_auc: 0.9816 - val_loss: 0.1620

```

Epoch 8/12

```

15/15 ————— 5s 312ms/step - accuracy: 0.9684 - auc: 0.9931 -
loss: 0.1043 - val_accuracy: 0.9398 - val_auc: 0.9767 - val_loss: 0.1851

```

```

MobileNetV2: {'params': 2422081, 'trainable_params': np.int32(164097),
'train_time_sec': 85.9}

```

==== Training EfficientNetB0 (feature extraction) =====

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5

```

16705208/16705208 ————— 0s 0us/step

```

Epoch 1/12

```

15/15 ————— 58s 2s/step - accuracy: 0.7911 - auc: 0.8832 -
loss: 0.4394 - val_accuracy: 0.8795 - val_auc: 0.9714 - val_loss: 0.2593

```

Epoch 2/12

```

15/15 ————— 4s 264ms/step - accuracy: 0.8945 - auc: 0.9626 -
loss: 0.2508 - val_accuracy: 0.9398 - val_auc: 0.9805 - val_loss: 0.1793

```

Epoch 3/12

```

15/15 ————— 4s 225ms/step - accuracy: 0.9304 - auc: 0.9824 -
loss: 0.1773 - val_accuracy: 0.9518 - val_auc: 0.9816 - val_loss: 0.1557

```

Epoch 4/12

```

15/15 ————— 6s 288ms/step - accuracy: 0.9388 - auc: 0.9848 -

```

```

loss: 0.1644 - val_accuracy: 0.9518 - val_auc: 0.9883 - val_loss: 0.1446
Epoch 5/12
15/15 ————— 4s 233ms/step - accuracy: 0.9325 - auc: 0.9865 -
loss: 0.1473 - val_accuracy: 0.9518 - val_auc: 0.9936 - val_loss: 0.1303
Epoch 6/12
15/15 ————— 4s 231ms/step - accuracy: 0.9578 - auc: 0.9913 -
loss: 0.1272 - val_accuracy: 0.9639 - val_auc: 0.9965 - val_loss: 0.1225
Epoch 7/12
15/15 ————— 4s 286ms/step - accuracy: 0.9641 - auc: 0.9954 -
loss: 0.1074 - val_accuracy: 0.9518 - val_auc: 0.9971 - val_loss: 0.1130
Epoch 8/12
15/15 ————— 4s 225ms/step - accuracy: 0.9789 - auc: 0.9975 -
loss: 0.0817 - val_accuracy: 0.9518 - val_auc: 0.9962 - val_loss: 0.1300
Epoch 9/12
15/15 ————— 5s 209ms/step - accuracy: 0.9726 - auc: 0.9949 -
loss: 0.0940 - val_accuracy: 0.9518 - val_auc: 0.9918 - val_loss: 0.1288
Epoch 10/12
15/15 ————— 5s 293ms/step - accuracy: 0.9705 - auc: 0.9967 -
loss: 0.0837 - val_accuracy: 0.9518 - val_auc: 0.9918 - val_loss: 0.1354
EfficientNetB0: {'params': 4213668, 'trainable_params': np.int32(164097),
'train_time_sec': 97.8}

```

==== Training Xception (feature extraction) =====

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/xception/xception_weights_tf_dim_ordering_tf_kernels_notop.h5
83683744/83683744 ————— 0s 0us/step
Epoch 1/12
15/15 ————— 51s 2s/step - accuracy: 0.7152 - auc: 0.8082 -
loss: 0.5575 - val_accuracy: 0.8313 - val_auc: 0.9443 - val_loss: 0.3245
Epoch 2/12
15/15 ————— 4s 236ms/step - accuracy: 0.8101 - auc: 0.8999 -
loss: 0.4034 - val_accuracy: 0.8675 - val_auc: 0.9612 - val_loss: 0.2351
Epoch 3/12
15/15 ————— 4s 238ms/step - accuracy: 0.8608 - auc: 0.9481 -
loss: 0.2905 - val_accuracy: 0.8434 - val_auc: 0.9575 - val_loss: 0.2294
Epoch 4/12
15/15 ————— 5s 277ms/step - accuracy: 0.8797 - auc: 0.9547 -
loss: 0.2706 - val_accuracy: 0.8795 - val_auc: 0.9639 - val_loss: 0.2531
Epoch 5/12
15/15 ————— 4s 235ms/step - accuracy: 0.9008 - auc: 0.9692 -
loss: 0.2255 - val_accuracy: 0.8916 - val_auc: 0.9723 - val_loss: 0.2278
Epoch 6/12
15/15 ————— 4s 225ms/step - accuracy: 0.8776 - auc: 0.9528 -
loss: 0.2866 - val_accuracy: 0.8434 - val_auc: 0.9738 - val_loss: 0.3064
Epoch 7/12
15/15 ————— 5s 319ms/step - accuracy: 0.8840 - auc: 0.9585 -
loss: 0.2617 - val_accuracy: 0.8916 - val_auc: 0.9659 - val_loss: 0.2151
Epoch 8/12
15/15 ————— 4s 222ms/step - accuracy: 0.8819 - auc: 0.9637 -
loss: 0.2437 - val_accuracy: 0.8916 - val_auc: 0.9685 - val_loss: 0.2192
Epoch 9/12
15/15 ————— 5s 244ms/step - accuracy: 0.9283 - auc: 0.9754 -
loss: 0.2047 - val_accuracy: 0.8554 - val_auc: 0.9677 - val_loss: 0.2186
Epoch 10/12

```

```
15/15 _____ 4s 266ms/step - accuracy: 0.9219 - auc: 0.9768 -  
loss: 0.1981 - val_accuracy: 0.9036 - val_auc: 0.9726 - val_loss: 0.2299  
Xception: {'params': 21123881, 'trainable_params': np.int32(262401),  
'train_time_sec': 89.4}
```

8. Evaluate All 5 Models on the Test Set

```
In [9]: def evaluate_model(model, ds):
        y_true, y_prob = [], []
        for x, y in ds:
            p = model.predict(x, verbose=0).ravel()
            y_prob.extend(p)
            y_true.extend(y.numpy().ravel())
        y_true = np.array(y_true); y_prob = np.array(y_prob)
        y_pred = (y_prob ≥ 0.5).astype(int)
        fpr, tpr, _ = roc_curve(y_true, y_prob)
        return {
            "y_true": y_true, "y_prob": y_prob, "y_pred": y_pred,
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred),
            "recall": recall_score(y_true, y_pred),
            "f1": f1_score(y_true, y_pred),
            "auc": auc(fpr, tpr),
        }

eval_results = {}
for name, model in trained_models.items():
    eval_results[name] = evaluate_model(model, test_ds)

comparison_rows = []
for name in MODEL_REGISTRY:
    r, e = results[name], eval_results[name]
    comparison_rows.append({
        "Model": name, "Parameters": r["params"], "Trainable Params":
r["trainable_params"],
        "Test Accuracy": round(e["accuracy"], 4), "Precision":
round(e["precision"], 4),
        "Recall": round(e["recall"], 4), "F1": round(e["f1"], 4), "AUC":
round(e["auc"], 4),
        "Training Time (s)": r["train_time_sec"],
    })

comparison_df = pd.DataFrame(comparison_rows).sort_values("F1",
ascending=False).reset_index(drop=True)
save_df_as_pdf(comparison_df, "04_model_comparison_table", title="Transfer
Learning Model Comparison (Feature Extraction)")
comparison_df
```

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.
<locals>.one_step_on_data_distributed at 0x798836b77e20> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to

https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.
<locals>.one_step_on_data_distributed at 0x798836b77e20> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

Saved: /content/outputs/04_model_comparison_table.pdf

Out[9]: Columns 1–8 of 10

	Model	Parameters	Trainable Params	Test Accuracy	Precision	Recall	F1
0	MobileNetV2	2422081	164097	0.9071	0.8608	0.9714	0.9128
1	EfficientNetB0	4213668	164097	0.9000	0.8590	0.9571	0.9054
2	ResNet50	23850113	262401	0.9000	0.8684	0.9429	0.9041
3	VGG16	14780481	65793	0.8714	0.8171	0.9571	0.8816
4	Xception	21123881	262401	0.8571	0.8125	0.9286	0.8667

Columns 9–10 of 10 · first column repeated

	AUC	Training Time (s)
0	0.9859	85.9
1	0.9486	97.8
2	0.9733	76.5
3	0.9680	63.3
4	0.9506	89.4

9. Select Top 2 Models & Fine-Tune

```
In [10]: top2 = comparison_df["Model"].tolist()[:2]
print("Top 2 models selected for fine-tuning:", top2)

# Fine-tuning: unfreeze the last N layers of the base network, retrain with a small LR
FT_EPOCHS = 8
FT_LR = 1e-5
UNFREEZE_LAST_N = 30

fine_tuned_models = {}
ft_histories = {}

for name in top2:
    print(f"\n===== Fine-tuning {name} =====")
    model = trained_models[name]
    base = model.layers[2] # Lambda(preprocess) → base model is layers[2]
    base.trainable = True
    for layer in base.layers[:-UNFREEZE_LAST_N]:
        layer.trainable = False

    compile_model(model, lr=FT_LR)
    es = callbacks.EarlyStopping(monitor="val_loss", patience=3,
restore_best_weights=True)
    hist = model.fit(train_ds, validation_data=val_ds, epochs=FT_EPOCHS,
callbacks=[es], verbose=1)

    fine_tuned_models[name] = model
    ft_histories[name] = hist.history
    eval_results[name] = evaluate_model(model, test_ds) # refresh with fine-tuned scores
    print(f"{name} fine-tuned test metrics: acc={eval_results[name]
['accuracy']:.4f} "
          f"f1={eval_results[name]['f1']:.4f} auc={eval_results[name]
['auc']:.4f}")

ft_rows = [{"Model": n, "Test Accuracy": round(eval_results[n]["accuracy"],
4),
            "F1": round(eval_results[n]["f1"], 4), "AUC":
round(eval_results[n]["auc"], 4)} for n in top2]
save_df_as_pdf(pd.DataFrame(ft_rows), "05_fine_tuning_results", title="Fine-
Tuning Results (Top 2 Models)")
```

Top 2 models selected for fine-tuning: ['MobileNetV2', 'EfficientNetB0']

===== Fine-tuning MobileNetV2 =====

Epoch 1/8

15/15 ————— 37s 1s/step - accuracy: 0.8966 - auc: 0.9765 -

loss: 0.2489 - val_accuracy: 0.9398 - val_auc: 0.9857 - val_loss: 0.1451

Epoch 2/8

15/15 ————— 4s 224ms/step - accuracy: 0.8776 - auc: 0.9657 -

```

loss: 0.2631 - val_accuracy: 0.9398 - val_auc: 0.9881 - val_loss: 0.1426
Epoch 3/8
15/15 ————— 4s 275ms/step - accuracy: 0.9304 - auc: 0.9812 -
loss: 0.2068 - val_accuracy: 0.9398 - val_auc: 0.9892 - val_loss: 0.1435
Epoch 4/8
15/15 ————— 4s 225ms/step - accuracy: 0.9262 - auc: 0.9800 -
loss: 0.1904 - val_accuracy: 0.9398 - val_auc: 0.9904 - val_loss: 0.1452
Epoch 5/8
15/15 ————— 3s 217ms/step - accuracy: 0.9093 - auc: 0.9744 -
loss: 0.2085 - val_accuracy: 0.9398 - val_auc: 0.9916 - val_loss: 0.1432
MobileNetV2 fine-tuned test metrics: acc=0.9214 f1=0.9252 auc=0.9888

```

```

===== Fine-tuning EfficientNetB0 =====
Epoch 1/8
15/15 ————— 56s 2s/step - accuracy: 0.9304 - auc: 0.9790 -
loss: 0.2071 - val_accuracy: 0.9518 - val_auc: 0.9959 - val_loss: 0.1161
Epoch 2/8
15/15 ————— 5s 300ms/step - accuracy: 0.9346 - auc: 0.9798 -
loss: 0.2050 - val_accuracy: 0.9518 - val_auc: 0.9950 - val_loss: 0.1188
Epoch 3/8
15/15 ————— 4s 218ms/step - accuracy: 0.8987 - auc: 0.9682 -
loss: 0.2374 - val_accuracy: 0.9518 - val_auc: 0.9948 - val_loss: 0.1212
Epoch 4/8
15/15 ————— 4s 223ms/step - accuracy: 0.9494 - auc: 0.9818 -
loss: 0.1935 - val_accuracy: 0.9518 - val_auc: 0.9953 - val_loss: 0.1220
EfficientNetB0 fine-tuned test metrics: acc=0.8929 f1=0.8980 auc=0.9490
Saved: /content/outputs/05_fine_tuning_results.pdf

```

```
Out[10]: '/content/outputs/05_fine_tuning_results.pdf'
```

10. Hyperparameter Tuning (top 2 models)

Grid over learning rate, dropout, and number of unfrozen layers.

```
In [11]: HP_GRID = [
    {"lr": 1e-3, "dropout": 0.2, "unfreeze": 0},
    {"lr": 1e-4, "dropout": 0.3, "unfreeze": 10},
    {"lr": 1e-5, "dropout": 0.5, "unfreeze": 20},
    {"lr": 1e-5, "dropout": 0.3, "unfreeze": 30},
]

hp_results = {}
for name in top2:
    hp_rows = []
    for cfg in HP_GRID:
        model, base = build_model(name, dropout=cfg["dropout"],
trainable_base=cfg["unfreeze"] > 0)
        if cfg["unfreeze"] > 0:
            for layer in base.layers[: -cfg["unfreeze"]]:
                layer.trainable = False
            compile_model(model, lr=cfg["lr"])
            es = callbacks.EarlyStopping(monitor="val_loss", patience=2,
restore_best_weights=True)
            hist = model.fit(train_ds, validation_data=val_ds, epochs=6,
callbacks=[es], verbose=0)
            val_acc = max(hist.history["val_accuracy"])
            hp_rows.append({**cfg, "val_accuracy": round(val_acc, 4)})
            print(name, cfg, "→ val_accuracy:", round(val_acc, 4))
        hp_results[name] = pd.DataFrame(hp_rows).sort_values("val_accuracy",
ascending=False)

for i, name in enumerate(top2):
    print(f"\n{name} hyperparameter results:")
    display(hp_results[name])
    save_df_as_pdf(hp_results[name], f"06_hyperparameter_tuning_{name}",
title=f"Hyperparameter Tuning — {name}")

MobileNetV2 {'lr': 0.001, 'dropout': 0.2, 'unfreeze': 0} → val_accuracy:
0.9518
MobileNetV2 {'lr': 0.0001, 'dropout': 0.3, 'unfreeze': 10} → val_accuracy:
0.8795
MobileNetV2 {'lr': 1e-05, 'dropout': 0.5, 'unfreeze': 20} → val_accuracy:
0.8313
MobileNetV2 {'lr': 1e-05, 'dropout': 0.3, 'unfreeze': 30} → val_accuracy:
0.8795
EfficientNetB0 {'lr': 0.001, 'dropout': 0.2, 'unfreeze': 0} → val_accuracy:
0.9759
EfficientNetB0 {'lr': 0.0001, 'dropout': 0.3, 'unfreeze': 10} →
val_accuracy: 0.9639
EfficientNetB0 {'lr': 1e-05, 'dropout': 0.5, 'unfreeze': 20} →
val_accuracy: 0.6747
EfficientNetB0 {'lr': 1e-05, 'dropout': 0.3, 'unfreeze': 30} →
val_accuracy: 0.6988
```

MobileNetV2 hyperparameter results:

	lr	dropout	unfreeze	val_accuracy
0	0.00100	0.2	0	0.9518
1	0.00010	0.3	10	0.8795
3	0.00001	0.3	30	0.8795
2	0.00001	0.5	20	0.8313

Saved: /content/outputs/06_hyperparameter_tuning_MobileNetV2.pdf

EfficientNetB0 hyperparameter results:

	lr	dropout	unfreeze	val_accuracy
0	0.00100	0.2	0	0.9759
1	0.00010	0.3	10	0.9639
3	0.00001	0.3	30	0.6988
2	0.00001	0.5	20	0.6747

Saved: /content/outputs/06_hyperparameter_tuning_EfficientNetB0.pdf

For the report: justify your final chosen configuration per model using the validation-accuracy table above — e.g. the smallest learning rate with the fewest unfrozen layers that still matches the best validation accuracy generalizes best and is cheaper to train.

11. Final Evaluation — Confusion Matrix, ROC, Curves

```
In [12]: BEST_MODEL_NAME = comparison_df["Model"].iloc[0] # update after reviewing
         fine-tuning + HP results
         best_model = fine_tuned_models.get(BEST_MODEL_NAME,
         trained_models[BEST_MODEL_NAME])
         best_eval = eval_results[BEST_MODEL_NAME]
         best_hist = ft_histories.get(BEST_MODEL_NAME, histories[BEST_MODEL_NAME])

         print(f"Best model: {BEST_MODEL_NAME}")
         report_text = classification_report(best_eval["y_true"],
         best_eval["y_pred"], target_names=["cat", "dog"])
         print(report_text)
         save_text_as_pdf(report_text, "07_classification_report",
         title=f"Classification Report — {BEST_MODEL_NAME}")

         cm = confusion_matrix(best_eval["y_true"], best_eval["y_pred"])
         fig, axes = plt.subplots(1, 3, figsize=(16, 4))

         axes[0].imshow(cm, cmap="Blues")
         axes[0].set_title("Confusion Matrix")
         for i in range(2):
             for j in range(2):
                 axes[0].text(j, i, cm[i, j], ha="center", va="center")
         axes[0].set_xticks([0, 1]); axes[0].set_xticklabels(["cat", "dog"])
         axes[0].set_yticks([0, 1]); axes[0].set_yticklabels(["cat", "dog"])
         axes[0].set_xlabel("Predicted"); axes[0].set_ylabel("Actual")
```

```
fpr, tpr, _ = roc_curve(best_eval["y_true"], best_eval["y_prob"])
axes[1].plot(fpr, tpr, label=f"AUC = {best_eval['auc']:.3f}")
axes[1].plot([0, 1], [0, 1], "--", color="gray")
axes[1].set_title("ROC Curve"); axes[1].set_xlabel("FPR");
axes[1].set_ylabel("TPR"); axes[1].legend()

axes[2].plot(best_hist["accuracy"], label="train acc")
axes[2].plot(best_hist["val_accuracy"], label="val acc")
axes[2].set_title("Accuracy Curves"); axes[2].legend()

plt.tight_layout()
save_fig(fig, "08_confusion_roc_accuracy")
plt.show()

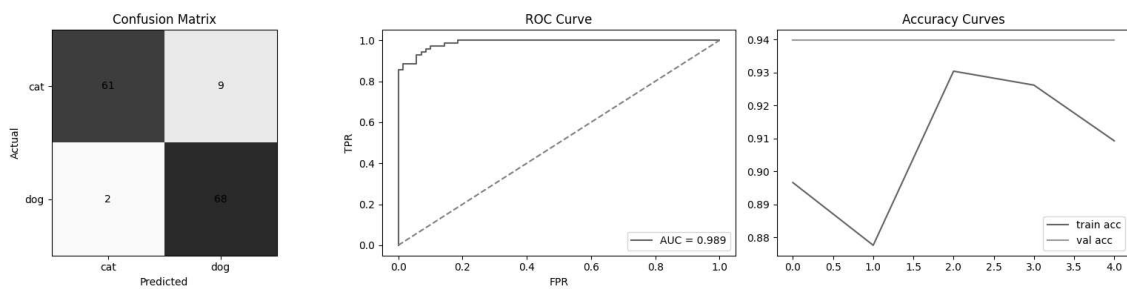
fig2 = plt.figure(figsize=(5, 4))
plt.plot(best_hist["loss"], label="train loss")
plt.plot(best_hist["val_loss"], label="val loss")
plt.title("Loss Curves"); plt.legend()
save_fig(fig2, "09_loss_curves")
plt.show()
```

Best model: MobileNetV2

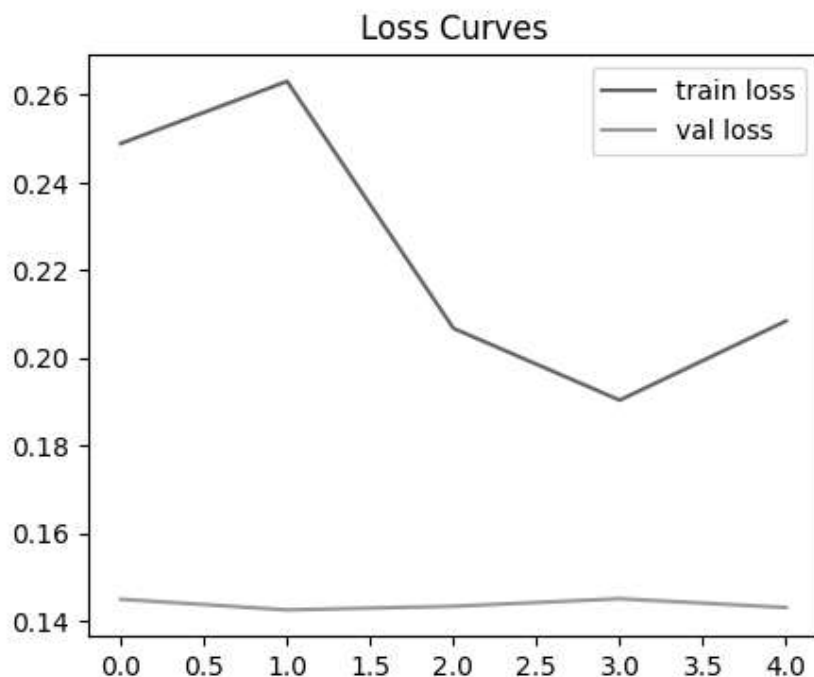
	precision	recall	f1-score	support
cat	0.97	0.87	0.92	70
dog	0.88	0.97	0.93	70
accuracy			0.92	140
macro avg	0.93	0.92	0.92	140
weighted avg	0.93	0.92	0.92	140

Saved: /content/outputs/07_classification_report.pdf

Saved: /content/outputs/08_confusion_roc_accuracy.pdf



Saved: /content/outputs/09_loss_curves.pdf



12. Error Analysis

```
In [13]: def gather_examples(ds, model, condition_fn, max_n=6):
    found = []
    for x, y in ds:
        p = model.predict(x, verbose=0).ravel()
        pred = (p ≥ 0.5).astype(int)
        for img, true_l, pred_l, prob in zip(x, y.numpy().ravel(), pred, p):
            if condition_fn(true_l, pred_l, prob):
                found.append((img.numpy().astype("uint8"), int(true_l),
int(pred_l), float(prob)))
            if len(found) ≥ max_n:
                return found
    return found

class_names = ["cat", "dog"]

def show_examples(examples, title, filename):
    n = len(examples)
    if n == 0:
        print(f"{title}: none found"); return
    fig = plt.figure(figsize=(3 * n, 3))
    for i, (img, t, p, prob) in enumerate(examples):
        plt.subplot(1, n, i + 1)
        plt.imshow(img)
        plt.title(f"T:{class_names[t]} P:{class_names[p]}\n{prob:.2f}",
        fontsize=9)
        plt.axis("off")
    plt.suptitle(title)
    plt.tight_layout()
    save_fig(fig, filename)
    plt.show()
```

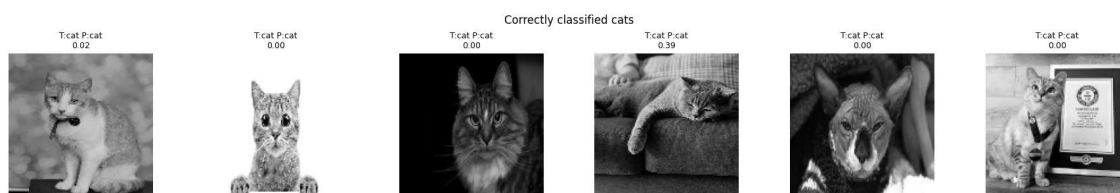
```

correct_cats = gather_examples(test_ds, best_model, lambda t, p, pr: t == 0
and p == 0)
correct_dogs = gather_examples(test_ds, best_model, lambda t, p, pr: t == 1
and p == 1)
cat_as_dog    = gather_examples(test_ds, best_model, lambda t, p, pr: t == 0
and p == 1)
dog_as_cat    = gather_examples(test_ds, best_model, lambda t, p, pr: t == 1
and p == 0)
low_conf      = gather_examples(test_ds, best_model, lambda t, p, pr: 0.4 ≤
pr ≤ 0.6)

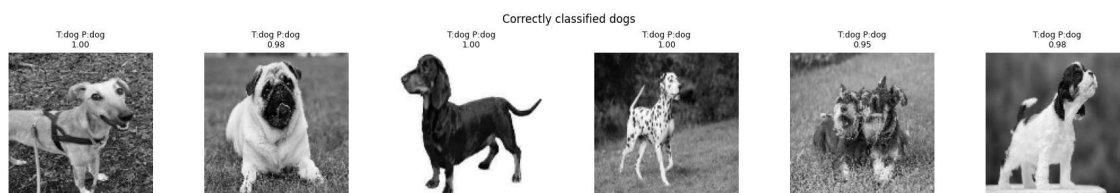
show_examples(correct_cats, "Correctly classified cats", "10_correct_cats")
show_examples(correct_dogs, "Correctly classified dogs", "11_correct_dogs")
show_examples(cat_as_dog, "Cat predicted as Dog", "12_cat_as_dog")
show_examples(dog_as_cat, "Dog predicted as Cat", "13_dog_as_cat")
show_examples(low_conf, "Low-confidence predictions (near 0.5)",
"14_low_confidence")

```

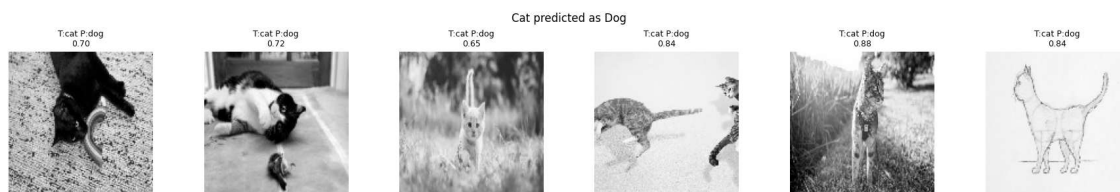
Saved: /content/outputs/10_correct_cats.pdf



Saved: /content/outputs/11_correct_dogs.pdf



Saved: /content/outputs/12_cat_as_dog.pdf



Saved: /content/outputs/13_dog_as_cat.pdf

Dog predicted as Cat

T:dog P:cat
0.34



T:dog P:cat
0.40



Saved: /content/outputs/14_low_confidence.pdf

Low-confidence predictions (near 0.5)

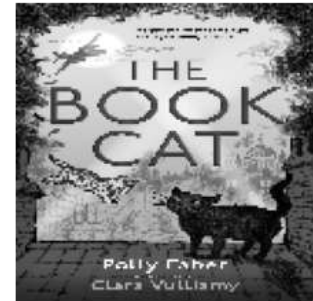
T:cat P:cat
0.41



T:cat P:dog
0.57



T:cat P:dog
0.51



13. Save the Final Model

```
In [14]: best_model.save("best_model.keras")
print("Saved best_model.keras – download it from the Colab file browser, "
      "or copy to Google Drive if you've mounted it.")

# Optional: mount Drive and copy
# from google.colab import drive
# drive.mount('/content/drive')
# shutil.copy("best_model.keras", "/content/drive/MyDrive/best_model.keras")
```

Saved best_model.keras – download it from the Colab file browser, or copy to Google Drive if you've mounted it.

13b. Package All PDF Outputs

Zips the `outputs/` folder (every PDF generated above) into one archive you can download in a single click, or copy to Drive.

```
In [15]: zip_path = shutil.make_archive("/content/outputs_pdfs", "zip", OUTPUT_DIR)
print("All PDFs saved in:", OUTPUT_DIR)
print(sorted(os.listdir(OUTPUT_DIR)))
print("Zipped archive:", zip_path)

# Download directly in Colab:
from google.colab import files
files.download(zip_path)

# Or copy the whole folder to Drive instead:
# from google.colab import drive
# drive.mount('/content/drive')
# shutil.copytree(OUTPUT_DIR, "/content/drive/MyDrive/cats_vs_dogs_outputs",
dirs_exist_ok=True)
```

```
All PDFs saved in: /content/outputs
['01_dataset_summary.pdf', '02_sample_image_grid.pdf',
'03_augmentation_examples.pdf', '04_model_comparison_table.pdf',
'05_fine_tuning_results.pdf', '06_hyperparameter_tuning_EfficientNetB0.pdf',
'06_hyperparameter_tuning_MobileNetV2.pdf', '07_classification_report.pdf',
'08_confusion_roc_accuracy.pdf', '09_loss_curves.pdf',
'10_correct_cats.pdf', '11_correct_dogs.pdf', '12_cat_as_dog.pdf',
'13_dog_as_cat.pdf', '14_low_confidence.pdf']
Zipped archive: /content/outputs_pdfs.zip
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.Javascript object>
```

14. Deployment

A ready-to-run Streamlit app (`app.py`) is provided alongside this notebook. To run it after downloading `best_model.keras` and `app.py` to the same folder:

```
pip install streamlit tensorflow pillow
streamlit run app.py
```

In Colab you can also tunnel it, e.g. with `pip install streamlit pyngrok` and `pyngrok`, but running it locally after downloading the model is simplest.